Applied Big Data Engineering Mini Project

Prepared By: EG/2020/4115 – Poornima K. N

EG/2020/4200 – Senapathy J. D

Applied Big Data Engineering Mini Project Report

E-Commerce Clickstream & Flash Sale Detection System Using Lambda Architecture

1. Introduction

This project presents the design and implementation of a real-time big data analytics pipeline for an e-commerce platform. The system simulates user clickstream activity in an online electronics store and processes the incoming data using a modern Lambda Architecture. The objective of the project is to analyze customer behavior in real time, identify products with high customer interest but low conversion rates, and generate daily analytical reports for business decision-making.

The system uses Apache Kafka for real-time data ingestion, Apache Spark Structured Streaming for stream processing, PostgreSQL as the serving and storage layer, Apache Airflow for orchestration and batch analytics, and Docker for containerized deployment. Synthetic user events are generated continuously using a Python producer script.

The project satisfies the requirements of the "E-Commerce Clickstream & Inventory Watch" scenario by implementing both real-time stream analytics and scheduled batch processing. The architecture supports scalable data processing, low-latency analytics, and automated reporting.

2. Project Objective

The main objective of this project is to build an end-to-end big data streaming pipeline capable of:

- Simulating real-time e-commerce user activity.
- Processing clickstream events using stream analytics.
- Detecting products with high interest but low purchase conversion.
- Generating automated reports and user segmentation analytics.
- Demonstrating the use of Lambda Architecture in a real-world scenario.

The system also aims to showcase how modern big data technologies can be integrated together to create a scalable and production-style analytics platform.

3. Selected Scenario

The selected scenario for this project is:

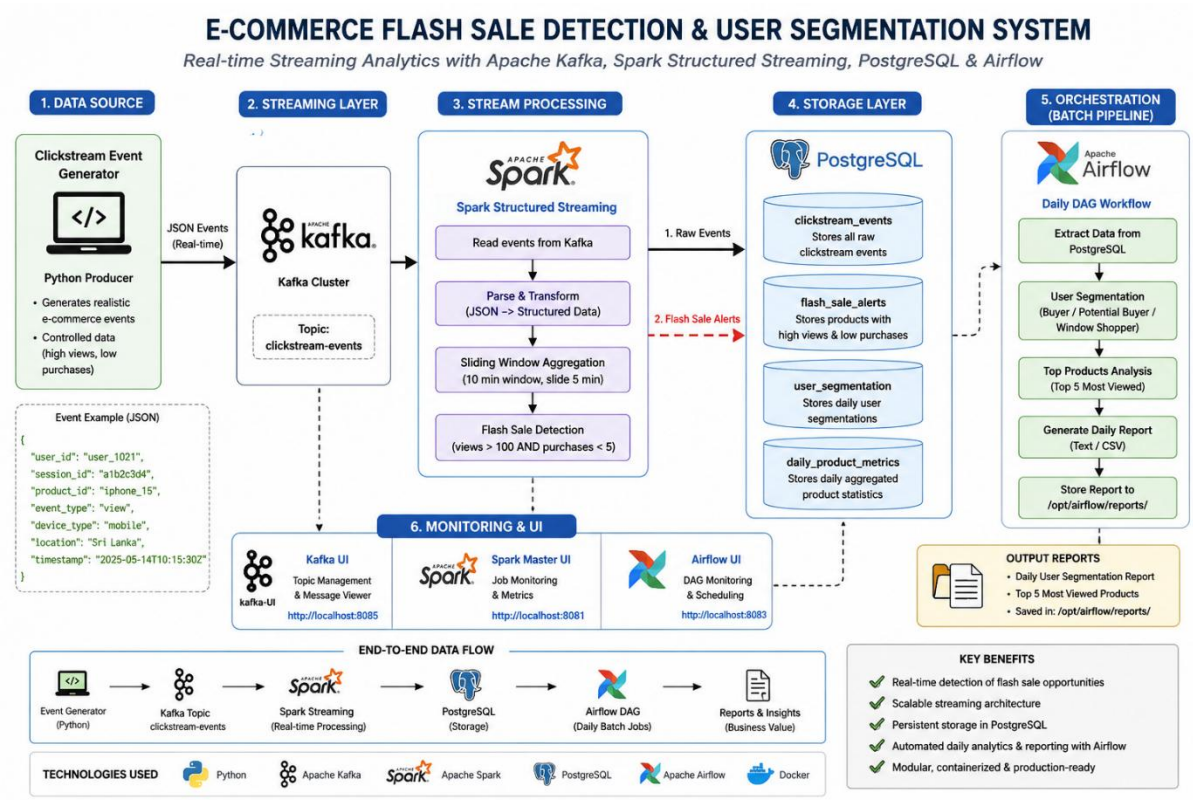
Scenario 3: E-Commerce Clickstream & Inventory Watch

The scenario describes an online electronics store that wants to monitor customer interactions with products in real time. The company requires a system that can detect products receiving a high number of views but very few purchases. Such products may require flash sales or promotional discounts to improve conversion rates.

Additionally, the system must generate scheduled analytical reports to classify users into behavioral segments and identify the most viewed products of the day.

4. System Architecture

The project follows the Lambda Architecture model, which combines real-time stream processing with batch analytics.



The architecture contains the following major layers:

4.1 Data Source Layer

A Python-based producer generates synthetic clickstream events. Each event contains:

- User ID
- Product ID
- Event Type
- Timestamp
- Device Type
- Location

The producer simulates realistic e-commerce behavior where most users only view products, fewer users add products to carts, and only a small percentage complete purchases.

4.2 Ingestion Layer

Apache Kafka is used as the real-time ingestion platform.

Kafka provides:

- High-throughput message streaming
- Scalability
- Fault tolerance
- Decoupling between producers and consumers

The producer continuously publishes events into the Kafka topic called:

```
clickstream-events
```

Kafka acts as the central event streaming backbone of the system.

4.3 Speed Layer (Real-Time Processing)

Apache Spark Structured Streaming is used as the speed layer of the Lambda Architecture.

Spark continuously reads streaming events from Kafka and processes them using sliding window aggregation.

The implemented analytics include:

- Counting product views
- Counting purchases
- Calculating conversion rates
- Detecting flash sale candidates

A flash sale alert is generated when:

- Product views exceed a threshold
- Purchases remain very low

This logic helps identify products with high customer interest but poor conversion performance.

4.4 Serving Layer

PostgreSQL is used as the serving and storage layer.

The database stores:

- Raw clickstream events

- Flash sale alerts
- User segmentation results
- Daily analytics metrics

PostgreSQL was selected because:

- It supports structured analytical queries
- It integrates easily with Spark and Airflow
- It provides reliable transactional storage
- It is simple to demonstrate and maintain

4.5 Batch Layer and Orchestration

Apache Airflow is used as the batch processing and orchestration tool.

Airflow performs:

- Daily user segmentation
- Product analytics generation
- Automated report generation

The Airflow DAG reads data from PostgreSQL and produces analytics reports in text format.

Users are segmented into:

- Buyers
- Potential Buyers
- Window Shoppers

5. Technology Stack Justification

Apache Kafka

Kafka was selected because it is one of the most widely used distributed event streaming platforms. It supports high-throughput real-time data ingestion and enables asynchronous communication between producers and consumers.

Benefits:

- Real-time streaming support
- Horizontal scalability
- Reliable event storage
- Fault-tolerant architecture

Apache Spark Structured Streaming

Spark Structured Streaming was selected due to its ability to process real-time streaming data efficiently using DataFrame APIs.

Benefits:

- Native Kafka integration
- Window-based analytics
- Scalability
- Event-time processing support
- High-performance distributed computation

Spark was ideal for implementing sliding window analytics and flash sale detection.

PostgreSQL

PostgreSQL was selected as the storage layer because it provides structured relational storage and supports analytical SQL queries.

Benefits:

- Reliable persistence
- SQL-based analytics
- Easy integration with Python and Airflow
- Good performance for medium-scale analytics

Apache Airflow

Airflow was selected to orchestrate scheduled workflows and reporting pipelines.

Benefits:

- DAG-based workflow orchestration
- Scheduling capabilities
- Monitoring interface
- Automation support

Airflow enabled automated daily reporting and user segmentation.

Docker

Docker was used to containerize the entire platform.

Benefits:

- Simplified deployment
- Environment consistency
- Service isolation
- Easy reproducibility

Docker Compose was used to manage all project services together.

6. Real-Time Processing Logic

The Spark streaming application continuously consumes clickstream events from Kafka.

The processing workflow includes:

1. Reading Kafka events
2. Parsing JSON messages
3. Converting timestamps
4. Applying watermarking
5. Sliding window aggregation
6. Flash sale detection
7. Writing analytics to PostgreSQL

7. Sliding Window Analytics

The system uses a sliding window model for stream aggregation.

Configuration:

- Window Size: 10 minutes
- Sliding Interval: 5 minutes

The sliding window continuously recalculates product analytics using the latest streaming events.

This approach enables:

- Near real-time insights
- Continuous analytics updates
- Timely flash sale detection

8. Flash Sale Detection

The system identifies products with:

- High customer interest
- Low conversion rates

Detection rule:

```
Views > 50 AND Purchases < 5
```

When this condition is met:

- A flash sale alert is generated
- The alert is stored in PostgreSQL
- The product becomes a candidate for discounts or promotions

This feature demonstrates practical business intelligence and event-driven analytics.

9. Event Time vs Processing Time

The project handles both event time and processing time concepts.

Event Time

Event time refers to the actual time at which a user action occurred.

Example:

- A customer viewed a product at 10:05 AM.

This timestamp is generated by the producer and stored inside the event payload.

Processing Time

Processing time refers to the time at which Spark processes the event.

Due to network delays or system latency, processing may occur later than the original event time.

Watermarking

Spark watermarking was implemented to handle delayed events.

Configuration:

```
.withWatermark("event_timestamp", "5 minutes")
```

This ensures:

- Late-arriving events are handled correctly
- Window calculations remain accurate
- Memory usage remains controlled

Handling event time correctly is critical in streaming analytics systems.

10. User Segmentation

The Airflow pipeline categorizes users based on their activity.

Segmentation logic:

User Type	Condition
Buyer	Completed purchases
Potential Buyer	Added products to cart
Window Shopper	Only viewed products

This segmentation helps businesses understand customer behavior and target marketing campaigns effectively.

11. Report Generation

The system generates automated analytics reports every two minutes during demonstrations.

The reports contain:

- Top viewed products
- Product details
- Conversion rates
- Flash sale recommendations
- Summary statistics

The generated reports provide actionable business insights.

Example metrics:

- Product views
- Purchases
- Add-to-cart counts
- Conversion percentages

12. Monitoring and User Interfaces

The project includes multiple monitoring interfaces:

Kafka UI

Used for:

- Topic inspection
- Message monitoring
- Stream verification

Spark UI

Used for:

- Monitoring streaming jobs
- Viewing execution stages
- Performance analysis

Airflow UI

Used for:

- DAG monitoring

- Workflow execution
- Scheduling visualization

These interfaces improve system observability and debugging capabilities.

13. Ethics and Data Governance

E-commerce analytics systems involve the collection and analysis of user behavior data. This introduces ethical and privacy concerns.

Potential concerns include:

- User tracking
- Behavioral profiling
- Privacy invasion
- Data misuse

To address these issues, organizations should implement proper data governance policies.

Recommended practices:

- User consent mechanisms
- Data anonymization
- Access control policies
- Encryption of sensitive data
- Limited data retention periods

Organizations must ensure compliance with privacy regulations such as GDPR and other data protection frameworks.

14. Challenges Faced

Several challenges were encountered during implementation.

Integration Complexity

Connecting Kafka, Spark, PostgreSQL, and Airflow together required careful configuration and dependency management.

Streaming Synchronization

Handling continuous streaming data while maintaining accurate window calculations required proper watermarking and event-time handling.

Docker Networking

Ensuring communication between containers required proper network configuration and service naming.

Realistic Data Simulation

Generating controlled low-conversion product behavior required custom producer logic and weighted event generation.

15. Future Improvements

The system can be extended further using:

- Machine learning recommendation systems
- Real-time dashboards using Grafana
- Cloud deployment using AWS or Kubernetes
- Distributed PostgreSQL clusters
- Fraud detection capabilities
- Real-time notification services

Future improvements would make the platform more scalable and enterprise-ready.

16. Conclusion

This project successfully implemented an end-to-end Lambda Architecture pipeline for real-time e-commerce analytics.

The system demonstrated:

- Real-time clickstream processing
- Sliding window analytics
- Flash sale detection
- Automated orchestration
- User segmentation
- Report generation

The integration of Kafka, Spark Structured Streaming, PostgreSQL, Airflow, and Docker created a scalable and professional analytics platform.

The project highlights how big data engineering technologies can be applied to solve practical business problems in modern e-commerce systems.